YTL*AI LABS

Z.AI

Refined Quality Assurance Testing Documentation—CliniFlow AI

Team Name : Dependency Free

Participant Name : Tang Shao Xian

Document Control **3**
FINAL ROUND (Execution, RCA & Hard Evidence) **4**
1. Test Execution & Evidence (Manual) 4
2. Automated Testing Pipeline 6
2.1 Unit Testing (Core Logic & Functions) 6
2.2 Integration & API Testing (System Modules) 8
3. Defect Log & Fix Traceability (Root Cause Analysis) 10
4. Known Issues & Deferred Technical Debt 13
5. Live Demo / UAT Risks 15

Document Control

Field	Detail	System Under Test (SUT)	UMHackathon 2026 — Team Dependency Free — CliniFlow AI
Participant	Tang Shao Xian	**Team Repo URL**	https://github.com/shaoxian04/ClinicFlowAI
CI/CD Pipeline	GitHub Actions — `.github/workflows/ci.yml` (frontend, backend, agent, gitleaks, Trivy, docker-build smoke test)	**Related Artifacts**	System Analysis Document (… Product Requirement Docu…

Objective:

The primary objective of this refined QATD is to demonstrate that **CliniFlow AI** — an AI-powered clinical workflow platform for Malaysian SME clinics — has been executed against the test plan declared in the preliminary round, that AI outputs (Pre-Visit symptom intake, SOAP generation, Post-Visit summary) are accurate, hallucination-resistant, and gated by mandatory doctor-in-the-loop review, and that every defect found has been logged, root-caused, and either fixed (with a verifiable Git commit hash) or transparently deferred as a known issue. Hard safety invariants — doctor confirmation before finalize, append-only PDPA audit log, JWT-derived patient identity, and Evaluator soft-block on CRITICAL findings — are verified by automated tests in CI on every push to `master`.

FINAL ROUND (Execution, RCA & Hard Evidence)

No evidence = No points.

1. Test Execution & Evidence (Manual)

Execution results for the test cases drafted in the Preliminary Round (§5 and §6.1), plus the new Evaluator soft-block test (TC-05) added after the cross-patient PHI incident on 2026-04-30. Visual proof for the four critical (P0) tests is linked in the *Proof of Execution* column.

Test Case ID	Test Type & Mapped Feature	Test Description	Expected Result	Actual Result	Status	Proof of Execution
TC-02	**Specific Case (Negative)** — Patient summary missing required clinical info (US-D05, US-P05)	System detects missing critical info (dose, frequency, duration) and blocks finalisation with a clarification prompt rather than producing an incomplete patient summary.	Publish blocked; clarification modal lists each missing field; HTTP response is **not** 500; app remains stable; audit_log records the attempted publish + clarification turn.	Clarification modal listed "Dosage not specified", "Frequency missing". Doctor filled the gaps and the publish succeeded on retry. App stable; no 5xx.	**Pass**	`frontend/e2e/post-visit-review-clarification.spec.ts`; screenshot `clarification-asked.png`.

TC-04	**NFR (Concurrency / Atomicity)** — Finalize must not split-brain (spec §5.5)	Two concurrent finalize requests on the same visit: exactly one succeeds; the other rolls back cleanly; no orphan `audit_log` row; `visit.status` and `medical_reports.is_finalized` stay consistent.	Both transactions roll back on induced audit_log INSERT failure. `visit.status` unchanged. `medical_reports.is_finalized = false`. No partial audit rows.	Both threads observed `BusinessException`; H2 state inspected — `visits.status` unchanged; `medical_reports.is_finalized = false`; `audit_log` count unchanged.	**Pass**	`backend/src/test/java/my/cliniflow/application/biz/visit/FinalizeAtomicityTest.java`; JUnit XML report uploaded to CI artifacts.
TC-AI-01	**AI Output Validation** — Pre-Visit symptom extraction	Send a symptom-intake turn to the GLM-backed Pre-Visit agent ("dry cough 3 days, mild night headache, no fever") and verify structured JSON extraction with no	JSON contains the cough + headache symptoms with correct qualifiers/duration; `fever: false` populated; no medication recommendations (Pre-Visit agent is read-only on clinical decisions).	Output matched the acceptance JSON exactly; fever correctly negated; no extra symptoms invented; Malay phrase "sakit kepala" mapped to `headache`.	**Pass**	Prompt + raw GLM response captured in `agent/tests/agents/test_pre_visit_agent.py` snapshot fixture.

		fabricated fields, correct `fever=false`, and graceful EN/BM code-switching.				

---

2. Automated Testing Pipeline

Three parallel jobs (`frontend`, `backend`, `agent`) plus three security/build jobs (`security` gitleaks, `trivy` Dockerfile scan, `docker-build` compose smoke) run on every push to `master`, `day*`, `feat/*`, `fix/*`, and on every pull request to `master`. CI definition: `.github/workflows/ci.yml` (introduced in commit `02ee0fc`, hardened in `d3fadfc`).

2.1 Unit Testing (Core Logic & Functions)

Unit tests isolate individual components/functions. External dependencies (DB, GLM, Neo4j, STT) are mocked.

Field	Detail	Test Runners	JUnit 5 + Mockito + AssertJ (Java 21 backend, run via Maven Surefire) · Pytest (Python 3.12 agent) · TypeScript `tsc --noEmit` • ESLint (frontend type/lint gate; Vitest unit suite tracked as deferred — see §4)

Targeted Components (backend)	`JwtService` (token signing/validation), `UserRepository` (RBAC lookups), `ReportAggregatorService` (post-visit aggregation), `VisitReadAppService` (visit read with ownership check), `ReferenceNumberDomainService` (per-day V-yyyy-MM-dd-NNNN counter — concurrency-tested), `EvaluatorFindingAcknowledgeDomainService` (override-with-reason write path), `OutboxPayloadBuilder` • `MessageTemplateRegistry` (WhatsApp templating), `JwtServiceTest`, `AuditWriterTest`	**Targeted Components (agent)**	`tool_spec` schema validation, `graph_queries` Cypher builders, `openai_client` plumbing, `pre_visit_extraction` JSON parsing, plus the full Evaluator suite: `dose_parser`, `dose_range`, `drug_drug_interactions`, `pregnancy_safety`, `hallucination`, `completeness`, `severity`, `findings_repo`, `orchestrator`
Files Covered	Backend: `backend/src/test/java/my/cliniflow/` — 50+ test classes. Agent: `agent/tests/` — 30+ test modules including the 14-file `agents/evaluator/` suite	**Execution Proof**	JaCoCo HTML report uploaded as `jacoco-report` CI artifact (retention 7 d) on every backend job run. Pytest output visible in the `agent` job log. Latest `master` run (08a6f50): all three lint/test jobs green; gitleaks clean; Trivy CRITICAL/HIGH = 0; `docker compose build --no-cache` succeeded.

Sample CI status (latest commit on `master` at submission time):

Job	Result	Duration	Notable Output
`backend — Maven test + coverage`	**Pass**	~3m 20s	`./mvnw verify -B` — 50+ JUnit classes green; JaCoCo report archived
`security — gitleaks`	**Pass**	~25 s	0 secrets detected (full git history scanned)
`Docker — compose build smoke test`	**Pass**	~4m 10s	`docker compose build --no-cache` succeeded for backend + frontend + agent

2.2 Integration & API Testing (System Modules)

Integration tests exercise real boundaries: Spring Boot controllers ↔ DB (H2 in PostgreSQL compatibility mode), Spring Boot ↔ Python agent over HTTP, agent routes ↔ Postgres + mocked GLM/Neo4j.

Field	Detail
Targeted Integrations	(a) `POST /api/previsit/sessions` → derives `patient_id` from JWT, inserts `visits`, writes `audit_log` (regression for cross-patient PHI leak — DEF-01). (b) `POST /api/auth/register` → `users` • `patients` • audit + WhatsApp consent. (c) `POST /api/visits/{id}/finalize` → Evaluator gate → `medical_reports` • audit, atomic. (d) Spring Boot ↔ Python agent contract via `ReportContractTest`. (e) Outbox drainer → WhatsApp Meta Cloud / Twilio sender selection. (f) Backend ↔ Postgres (Supabase) connection pool through pgbouncer.
Execution Proof	Maven Surefire/Failsafe XML reports per integration test class; Playwright HTML report (videos, screenshots, network log) attached to the CI run; k6 JSON output for `/agents/visit/generate` (see TC-03).

Notable integration test outcomes:

Integration Test ID	Integration Point	Test Scenario	Expected Outcome	Actual Result	Status
IT-02	`POST /api/previsit/sessions/{id}/turns` ownership check	Patient B attempts to send a turn on Patient A's visit; expect HTTP 40300 `FORBIDDEN`, not silent execution.	HTTP 403 with code `40300`; no row written to `agent_turns`.	HTTP 403 in 15 ms; `agent_turns` row count unchanged. (Test `turn_on_another_patients_visit_is_forbidden`.)	**Pass**
IT-04	`POST /api/visits/{id}/finalize` after override-with-reason	Doctor acknowledges each CRITICAL finding via `ApproveOverrideDialog` with a written reason, retries finalize.	HTTP 200; `medical_reports.is_finalized = true`; `evaluator_findings.acknowledged_at` populated; two `audit_log` rows (`OVERRIDE` • `FINALIZE`).	HTTP 200 in 410 ms; all DB invariants satisfied.	**Pass**
IT-06	WhatsApp outbox drainer end-to-end	Append a `BOOKING_CONFIRMED` event; outbox drainer picks it up, builds the templated payload, sends via Meta Cloud sender	One `OutboxRow` marked `SENT`; one templated request to Meta Cloud with the right template name; idempotency key respected on retry.	OK; retry on simulated 503 did not produce a duplicate send.	**Pass**

		(mocked HTTP).			

3. Defect Log & Fix Traceability (Root Cause Analysis)

The following defects were discovered during execution and have been fixed on `master`. Each row links to the exact Git commit hash that closed the defect — these hashes cannot be fabricated and can be verified in the project's Git history.

Bug ID	Severity	Bug Description	Steps to Reproduce	Console / Error Log Snippet	Root Cause Analysis (Why did it break?)	Fix Commit Hash / PR Link
DEF-02	**High**	Backend boot loop — Hibernate validation rejected `UserModel` because `consent_given_at` was missing in the live DB even though the migration file existed in source.	1. Add `V4__add_consent_given.sql`. 2. Run `docker compose up --build -d`. 3. Backend container crash-loops with `missing column consent_given_at`.	`org.hibernate.tool.schema.spi.SchemaManagementException: Schema-validation: missing column [consent_given_at] in table [users]`	Two compounding issues: (a) Docker layer cache invalidation: regular `--build` reused the `COPY src ./src` layer and shipped the JAR without the new migration file. (b) `application.yml` had `spring.autoconfigure.exclude: FlywayAutoConfiguration` which silently no-op'd the `spring.flyw`	Removed Flyway entirely (incompatible with Supabase pgbouncer); schema is now managed manually via the Supabase SQL editor; `spring.jpa.hibernate.ddl-auto: none`. CI now runs `docker compose build --no-cache` as a smoke test (commit `d3fadfc`) so this exact failure mode

					`ay.enabled: true` line, so the migration would not have run anyway. (c) `ddl-auto: validate` then blocked startup. See post-mortem 2026-04-22-backend-boot-and-schema.md.	is caught before merge.
DEF-04	**Medium**	Frontend `apiPost` threw `"empty response data"` for legitimately void endpoints (consent submit, mark-as-read), surfacing a misleading red toast even though the backend operation succeeded.	1. Submit the WhatsApp consent toggle on `/portal/profile`. 2. Observe red toast "empty response data" while the consent state actually flipped on next refresh.	Browser console: `Error: empty response data` from `lib/api.ts::apiPost`	`apiPost` had a hard guard `if (envelope.data == null) throw new Error("empty response data")`. Endpoints returning `{ "code": 0, "data": null }` (a legitimately void success) hit the catch path, even though the actual operation had succeeded.	Added `apiPostVoid(path, body?)` to `lib/api.ts` (same logic minus the null guard) for endpoints that intentionally return `data: null`. Refactored 6 callers to use the new helper. Tracked under PR #11.

DEF-06	**Low**	Three frontend components (`SkeletonGrid`, `EmptyState`, `VisitRow`) were used with wrong prop names in `doctor/finalized/page.tsx`, producing successive TypeScript build failures inside Docker (~45 s each iteration to discover the next error).	1. `docker compose build frontend`. 2. Build fails with prop-name mismatch. 3. Fix one. 4. Repeat ×3.	`Type error: Type '{ rows: number }' is not assignable to type 'IntrinsicAttributes & SkeletonGridProps'. Property 'rows' does not exist on type … Did you mean 'count'?`	JSX written without first reading the component's props type. TypeScript caught the mismatch but only at Docker build time, not during local editing because the project does not run `tsc --watch` locally by default.	Fixed in the same Day 3 PR; codified the rule "Read the component's props type before using it" in the post-mortem and CLAUDE.md. CI gain (commit `bc34d85`): wiring H2 in-memory DB so `mvn verify` runs `tsc --noEmit` equivalent on every push.

4. Known Issues & Deferred Technical Debt

It is normal to ship with minor bugs and architectural trade-offs. The items below are known, transparently tracked, and intentionally deferred. Each will be opened as a GitHub Issue before judging.

Bug ID	Description & Impact	Reason for Deferral	Tracking
KI-02	**Frontend unit-test framework not yet wired (Vitest).** Test coverage measurement currently excludes the frontend; reported coverage of ~74% is backend + agent only. The preliminary QATD declared an 81.2% target for merge to `master`.	Hackathon scope was capped at type-checking + ESLint + Playwright E2E for the frontend layer. Adding a Vitest suite is mechanical but needs ~1–2 days of targeted testing to be meaningful; not justified for the demo cycle.	Tracked as TODO in `frontend/package.json` script comments; will become `gh issue` "Wire Vitest for frontend unit tests + raise overall coverage to ≥81.2%".

KI-04	**Schema drift risk — Flyway is removed; schema is managed manually via the Supabase SQL editor.** A new migration that exists in `backend/src/main/resources/db/migration/` is documentation only; it is not auto-applied. A developer who forgets to run the SQL in Supabase will see Hibernate startup errors only at runtime.	Flyway is incompatible with Supabase pgbouncer (the PostgreSQL transaction-pooled connection). Re-introducing Flyway requires a session-pooled side connection or a separate migration runner — out of scope for the hackathon.	To be tracked as `gh issue` "Re-introduce a migration runner compatible with Supabase pgbouncer". Workaround documented in CLAUDE.md "Database setup".
KI-06	**Evaluator drug knowledge graph is seeded from a small static fixture (~120 drug nodes).** The MVP does not pull from an authoritative drug database (e.g. FDA / MIMS). DDI and dose-range checks therefore have known coverage gaps for less-common drugs.	Licensing of a real drug knowledge base is a commercial discussion outside hackathon scope. The Evaluator is *correct on the drugs it knows* — and the doctor-in-the-loop gate is the safety backstop for unknowns.	Documented in `docs/details/data-model.md` §Neo4j drug knowledge graph; tracked as a roadmap item rather than a defect.

Proactively documenting these items demonstrates engineering maturity and risk awareness. We have not hidden any defect that we are aware of; if a judge finds an additional issue, please flag it and we will treat it as a priority.

---

5. Live Demo / UAT Risks

Every prototype has operational limits. The following are the fragile parts of the system that the judging panel should know about before testing the live demo. Following these instructions ensures a fair evaluation within documented parameters.

Risk #	Boundary / Operational Instruction	Risk 1	**Use the seeded demo accounts for the doctor / staff / admin walkthrough. Credentials: `doctor@demo.local` / `password`, `staff@demo.local` / `password`, `admin@demo.local` / `password`. Registering a brand-new patient + then immediately attempting the doctor flow on that patient *without* the receptionist check-in step will land in the empty-waiting-list state — by design, but easy to mistake for a bug.**
Risk 2	**Audio upload size limit is 25 MB / ~30 min per consultation.** The backend enforces HTTP 413 above this. Please do not upload longer recordings during the demo — the rejection is correct behaviour but will look like an error in a 90-second pitch slot.	Risk 3	**SOAP generation latency on the free GLM tier is 3–6 s p95.** Please click *Generate Report* once and wait. Repeated clicks will queue extra GLM requests and may trip the per-doctor rate limiter (Resilience4j) for ~30 s.
Risk 4	**Browser compatibility — optimised for Google Chrome (and Edge).** Safari has known issues with the `MediaRecorder` API used in the voice-input mode of the consultation capture; please use Chrome to demo the voice-recording path. Mobile browsers are out of scope for	Risk 5	**Evaluator soft-block requires acknowledgement with a written reason.** Triggering a CRITICAL DDI (e.g. warfarin + ibuprofen) will block finalize until the doctor opens *Approve & Override* and types a reason for each finding. This is the intended safety design, not a bug — please allow ~10 s in the demo flow for this dialog if you choose a test case that triggers a CRITICAL finding.

	the MVP (no native iOS/Android app — see PRD §7).		
Risk 6	**Frontend talks to Spring Boot only.** Nginx does not expose `:8000` (Python agent) or the Supabase JS client to the browser. A network panel showing only `:80/api/*` calls is *correct* — the agent is reached through Spring Boot's server-to-server adapter, not directly from the browser. This is enforced architecturally for PHI safety.	**Risk 7**	**Single-instance Docker Compose deployment for the demo.** The stack is not horizontally scaled — the demo environment can comfortably handle 5 concurrent users. Load tests in §1 (TC-03) used a separate isolated environment; please do not run the load script against the live demo URL.